

ELO329 - Diseño y Programación Orientados a Objetos

Manejo de excepciones en C++

Agustín J. González
Patricio Olivares R.

Universidad Técnica Federico Santa María

Propósito de la clase

Hasta ahora hemos usado funciones, clases, memoria dinámica, operadores, herencia y métodos virtuales.

El siguiente problema aparece cuando una función detecta un error, pero no es el mejor lugar para decidir qué hacer.

Las excepciones permiten comunicar situaciones anormales desde donde se detectan hacia donde pueden manejarse.

Ruta

En esta clase seguiremos una historia simple:

1. Una operación detecta un error.
2. El flujo normal no debería llenarse de chequeos.
3. Se lanza una excepción con `throw`.
4. Otro bloque decide cómo recuperarse con `try` y `catch`.
5. El error se vuelve más expresivo usando clases de excepción.

El problema

Queremos insertar un valor en una posición de un `vector`.

```
void Insert(std::vector<int> &array, int index, int value)
{
    array[index] = value;
}
```

La versión simple asume que `index` siempre es válido.

El problema real es que esa condición puede fallar.

Error detectado lejos del manejo

La función `Insert` puede saber si el índice está fuera de rango.

Pero no siempre sabe qué decisión tomar:

- volver a pedir el índice.
- registrar el error.
- terminar la operación.
- mostrar un mensaje al usuario.
- dejar que una función superior decida.

Detectar el error y manejar el error no siempre ocurre en el mismo lugar del programa.

Idea central

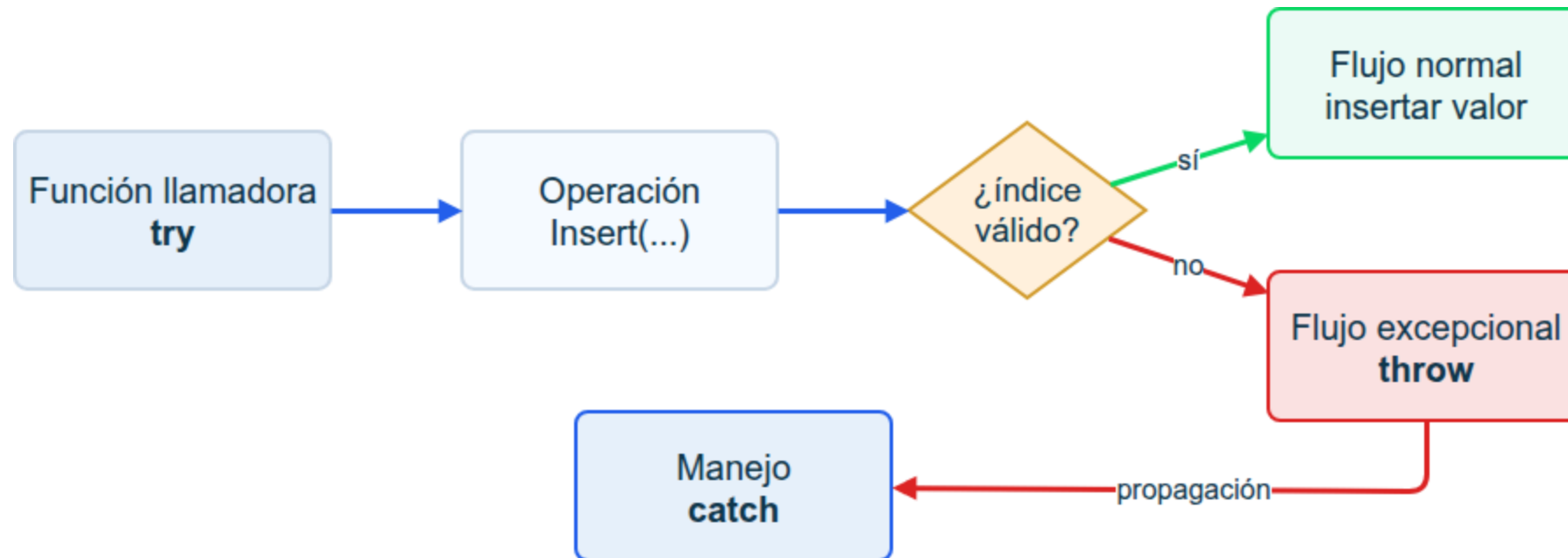
El manejo de excepciones separa dos caminos:

Camino	Qué representa
Flujo normal	caso esperado del programa
Flujo excepcional	situación anormal que interrumpe ese camino

`throw` comunica el problema.

`try` y `catch` delimitan dónde se intenta ejecutar y dónde se maneja el error.

Flujo de una excepción



El punto que detecta el error no siempre es el punto que lo maneja.

Lanzar una excepción

```
void Insert(std::vector<int> &array, int index, int value)
{
    if (index < 0 || index >= array.size()) {
        throw std::string("Index out of bounds in Insert()");
    }

    array[index] = value;
}
```

Cuando se ejecuta `throw`, la función no continúa normalmente.

El control vuelve hacia algún bloque que pueda capturar la excepción.

Capturar una excepción

```
try {  
    std::cout << "Ingrese un indice: ";  
    std::cin >> index;  
  
    Insert(scores, index, value);  
    std::cout << "Insercion exitosa\n";  
} catch (std::string &s) {  
    std::cout << s << std::endl;  
}
```

El bloque `try` contiene código que puede fallar.

El bloque `catch` recibe el tipo de excepción que sabe manejar.

Qué viaja con `throw`

En C++ se puede lanzar:

- un valor escalar, como `int` o `char`.
- un objeto de biblioteca, como `std::string`.
- un objeto definido por el programador.
- un objeto derivado de `std::exception`.

Que el lenguaje lo permita no significa que todos los tipos sean una buena decisión de diseño.

Tipos de excepción



El tipo comunica qué problema ocurrió y permite capturar solo lo que se sabe manejar.

Varias capturas

```
try {  
    DoOneThing();  
    DoAnother();  
    DoSomethingElse();  
} catch (int value) {  
    std::cout << "int exception";  
} catch (char value) {  
    std::cout << "char exception";  
} catch (...) {  
    std::cout << "default exception";  
}
```

`catch(...)` captura cualquier excepción no manejada antes.

Debe quedar al final, porque es el caso más general.

Especificar lo que una función lanza

En material C++ clásico puede aparecer:

```
float myFunction(char param) throw(int);
```

También se puede declarar que una función no lanza excepciones:

```
int myFunction(int param) throw();    // C++98  
int myFunction(int param) noexcept;  // C++11
```

Si no se declaraba nada especial, la función podía lanzar cualquier tipo:

```
int myFunction(int param);
```

Las especificaciones dinámicas `throw(...)` pertenecen al C++ clásico. En C++ moderno se usa `noexcept`.

Por qué usar clases

Lanzar un `string` funciona para ejemplos simples.

Pero una clase de excepción permite expresar mejor el problema:

- el tipo identifica la categoría del error.
- el objeto puede llevar un mensaje.
- el receptor puede capturar solo lo que sabe manejar.
- la jerarquía puede organizar errores relacionados.

Excepción propia mínima

```
class RangeException {  
};
```

Esta clase no tiene datos, pero su tipo ya comunica algo:

ocurrió un error de rango.

```
if (index < 0 || index >= array.size()) {  
    throw RangeException();  
}
```

Captura por tipo

```
try {  
    Insert(scores, index, value);  
    std::cout << "Insercion exitosa\n";  
} catch (RangeException &) {  
    std::cout << "A range exception occurred.\n";  
}
```

La captura ya no depende del texto del mensaje.

Depende del tipo de problema.

Esto acerca el manejo de errores al diseño orientado a objetos.

Múltiples excepciones propias

```
try {  
    DoOneThing();  
    DoAnother();  
    DoSomethingElse();  
} catch (RangeException &) {  
    std::cout << "A range exception occurred.\n";  
} catch (OpenFileError &) {  
    std::cout << "Cannot open file.\n";  
}
```

Cada `catch` representa un problema que el llamador sabe manejar.

Mensaje dentro de la excepción

```
class RangeException {  
public:  
    RangeException(const std::string &msg)  
        : m_msg(msg) {}  
  
    std::string GetMsg() const  
    {  
        return m_msg;  
    }  
  
private:  
    std::string m_msg;  
};
```

Ahora el tipo indica la categoría y el objeto transporta detalle.

Lanzar una excepción con mensaje

```
void Insert(std::vector<int> &array, int index, int value)
{
    if (index < 0 || index >= array.size()) {
        throw RangeException("Index out of bounds in Insert()");
    }

    array[index] = value;
}
```

La función mantiene claro su flujo normal.

El manejo específico queda fuera.

`std::exception`

La biblioteca estándar ofrece `std::exception`.

`std::exception` define `what()` como un método virtual:

```
virtual const char *what() const noexcept; // C++11 en adelante
// En C++98:
// virtual const char *what() const throw();
```

Por eso una clase derivada puede redefinir `what()` y entregar su propio mensaje de error.

La firma debe coincidir con la del método base. `override` hace explícita esa redefinición.

std::exception

Una clase propia puede derivar de ella y redefinir `what()` :

```
#include <exception>

class MiExcepcion : public std::exception {
public:
    explicit MiExcepcion(const std::string &msg)
        : mensaje(msg) {}

    const char *what() const noexcept override
    {
        return mensaje.c_str();
    }

private:
    std::string mensaje;
};
```

Usar `what()`

```
try {  
    throw MiExcepcion("Division por cero");  
} catch (const MiExcepcion &e) {  
    std::cout << e.what() << std::endl;  
}
```

`what()` entrega una descripción textual del error.

`noexcept` indica que `what()` no debería lanzar otra excepción.

Reenvío de una excepción

A veces una función puede detectar o registrar el problema, pero no resolverlo.

```
try {  
    Insert(scores, index, value);  
} catch (RangeException &e) {  
    throw;  
}
```

`throw;` relanza la excepción actual sin crear una nueva.

Así otra función más arriba puede manejarla.

Excepción desconocida

Si una excepción no fue capturada por casos específicos, se puede usar:

```
try {  
    Example();  
} catch (...) {  
    std::cout << "Caught unknown exception\n";  
}
```

Esto evita que el programa ignore completamente un error inesperado.

Pero también pierde información específica del tipo de excepción.

Diferencias con Java

El mecanismo general es parecido: lanzar, propagar y capturar.

Pero en C++:

- se pueden lanzar tipos básicos u objetos.
- no existe el mismo modelo de excepciones verificadas de Java.
- `std::exception` es una base común, pero no es obligatorio usarla.
- la responsabilidad de recursos sigue siendo importante.

Excepciones y recursos

Cuando una excepción interrumpe el flujo normal, el programa salta a otro punto.

Eso vuelve peligroso dejar recursos a medio manejar.

Ejemplo de riesgo:

```
Recurso *r = new Recurso();  
operacionQuePuedeLanzar();  
delete r;
```

Si `operacionQuePuedeLanzar()` falla, `delete r` podría no ejecutarse.

Criterio de uso

Use excepciones para situaciones anormales que interrumpen la operación.

No conviene usarlas para:

- control normal de ciclos.
- validaciones esperadas en cada entrada.
- reemplazar condiciones simples.
- ocultar errores de diseño.

Una excepción debe hacer más claro el programa, no más difícil de seguir.

Ideas clave

- `throw` comunica una situación excepcional.
- `try` delimita código que podría lanzar.
- `catch` maneja tipos específicos de error.
- `catch(...)` es una red general, no un reemplazo de buen diseño.
- clases propias permiten expresar mejor el problema.
- `std::exception` y `what()` entregan una interfaz común.
- `noexcept` declara funciones que no deberían lanzar.

Referencias

- C++ reference, exception handling:
<https://en.cppreference.com/w/cpp/language/exceptions>
- C++ reference, `std::exception` :
<https://en.cppreference.com/w/cpp/error/exception>